

Flowcharts and Visualization

Lesson Objectives

By the end of this lesson, students should be able to:

- Understand what a **flowchart** is and why it's used.
- Draw flowcharts to represent algorithms visually.
- Translate **pseudocode** into a **flowchart**.
- Use digital tools such as **draw.io** and **Lucidchart** to create algorithm diagrams.
- Present their algorithm visually for documentation or debugging.

1 What is a Flowchart?

A **flowchart** is a **diagrammatic representation** of an algorithm.

It shows the **sequence of operations** using **symbols** and **arrows** that indicate the flow of control.

Flowcharts are useful for:

- Visualizing algorithm logic
 - Debugging programs
 - Communicating program flow clearly
 - Documenting processes
-

2 Common Flowchart Symbols

Symbol	Meaning	Description
◆ Oval	Start / End	Marks the beginning or end of a process.
◆ Parallelogram	Input / Output	Used for data entry or result display.
■ Rectangle	Process	Represents an action or calculation step.
▲ Diamond	Decision	Used for IF, ELSE, or condition checks.
■ Arrow	Flow Line	Indicates the direction of flow between steps.
◆ Connector	Jump	Used to connect separated parts of the flowchart.

3 Rules for Drawing Flowcharts

1. Use a **top-to-bottom** or **left-to-right** flow.
2. Connect all symbols with **arrows**.
3. Keep symbols aligned and spaced evenly.
4. Each flowchart must have a **Start** and **End** symbol.
5. Use **decision diamonds** for conditions (Yes/No).
6. Avoid crossing lines — use connectors if needed.

4 Example 1: Simple Flowchart (Sum of Two Numbers)

◆ Pseudocode

BEGIN

INPUT A, B

SUM $\leftarrow$ A + B

OUTPUT SUM

END

◆ Flowchart Description

1. **Start** $\rightarrow$ (Oval)
2. **Input A, B** $\rightarrow$ (Parallelogram)
3. **Process: SUM = A + B** $\rightarrow$ (Rectangle)
4. **Output SUM** $\rightarrow$ (Parallelogram)
5. **End** $\rightarrow$ (Oval)

💡 (Diagram suggestion: 5 symbols vertically connected by arrows.)

5 Example 2: Flowchart for Finding the Largest Number

◆ Pseudocode

BEGIN

INPUT A, B, C

IF A > B AND A > C THEN

OUTPUT "A is largest"

ELSE IF B > C THEN

OUTPUT "B is largest"

```
ELSE
    OUTPUT "C is largest"
ENDIF
END
```

◆ Flowchart Description

1. Start
2. Input A, B, C
3. Decision: Is $A > B$ and $A > C$?
 - Yes → Output "A is largest"
 - No → Next decision: Is $B > C$?
 - Yes → Output "B is largest"
 - No → Output "C is largest"
4. End

✿ (This flowchart uses two diamonds for nested IF conditions.)

6 Tools for Drawing Flowcharts

a) draw.io (diagrams.net)

- Free and web-based
- Drag-and-drop flowchart symbols
- Export to PNG, SVG, or PDF
- Integrates with Google Drive and GitHub

b) Lucidchart

- Cloud-based diagramming tool
- Real-time collaboration
- Built-in templates for flowcharts and algorithms

c) Alternatives

- Microsoft Visio
 - Creately
 - Canva (simple flow diagrams)
-

7 Lab Activity

Objective:

Convert an existing pseudocode algorithm into a visual flowchart.

Tasks:

1. Choose any algorithm from Weeks 7–9 (e.g., Factorial, Coin Change, or Shortest Path).
 2. Draw its flowchart using **draw.io** or **Lucidchart**.
 3. Label all shapes clearly and maintain a clean layout.
 4. Export the diagram as **PDF** or **PNG**.
 5. Submit your diagram and a short description explaining your logic.
-

8 Git Task

1. Create a new branch:
2. `git checkout -b flowchart_visuals`
3. Add your exported flowchart image:
4. `git add factorial_flowchart.png`
5. `git commit -m "Added flowchart visualization for factorial algorithm"`
6. `git push origin flowchart_visuals`
7. Include a short README explaining:
 - What the algorithm does.
 - Which flowchart tool you used.
 - What decisions or loops are shown.

